

SOUGATA SAHA
IT-A2-037 OOP ASSIGNMENT 1

Assignment 5

C++ style Input/Output

1. Write a program that accepts two integers from keyboard, adds them and prints their values. Use cin and cout.

```
#include<iostream>
using namespace std;
```

```
int main(){
    int x,y,z;
    cin>>x>>y;
    z = x+y;
    cout<<z;
    return 0;
}
```

2. Create a factorial table using cout as follows:

```
1! = 1
2! = 2
3! = 6
...
6! = 720
```

```
#include<iostream>
using namespace std;
```

```
int main(){
    int fact = 1;
    for(int i=1;i<=6;i++){
        fact = fact*i;
        cout<<i<<"!="<<fact<<endl;
    }
    return 0;
}
```

Flexible Variable Declaration

1. Write a program to print 1 to 10 using a for loop. Declare the loop variable inside the for loop. Check the scope of this variable.

```
#include<iostream>
using namespace std;
```

```
int main(){
    for(int i=1;i<=10;i++){
        cout<<i<<" ";
    }
}
```

```

    }
    //cout<<i; ERROR
    return 0;
}

```

2. Write a program to display Celsius to Fahrenheit conversion table using a for loop. Consider only 0° to 100° Celsius. Declare variables when they are used for the first time.

```

#include<iostream>
using namespace std;

int main() {
    for(int i=0;i<=100;i++){
        float f = i*1.8+32;
        cout<<i<<"C="<<f<<"F"<<endl;
    }
    return 0;
}

```

Constants

1. Write a program that defines a constant PI and takes radius of a circle from keyboard and prints area of that circle.

```

#include<iostream>
using namespace std;

int main(){
    const float pi = 3.14;
    float r;
    cin>>r;
    float area = pi*r*r;
    cout<<area;
    return 0;
}

```

2. Write a function that takes an integer and returns the factorial of that number. Declare function parameter as const. Call the function with some argument from main function, store the result and print it.

```

#include<iostream>
using namespace std;

int fact(const int& n){
    if(n==0) return 1;
    else return n*fact(n-1);
}

int main(){
    int n;

```

```

    cin>>n;
    int f = fact(n);
    cout<<f;
    return 0;
}

```

Reference

1. Write a function swap() that takes two integer arguments and interchanges the values of those arguments using reference. Now in the main function, instantiate two integer variables with some values. Print their values. Call the swap function with these variables. Finally print the values of those variables. Check the result.

```

#include<iostream>
using namespace std;

void Swap(int& x,int& y){
    int temp = x;
    x = y;
    y = temp;
}
int main(){
    int a=2,b=3;
    cout<<"Before a:"<<a<<" b:"<<b<<endl;
    Swap(a,b);
    cout<<"After a:"<<a<<" b:"<<b<<endl;
    return 0;
}

```

2. Now write another function swap() that takes two strings (character array) and interchanges them without reference parameters. Test this function using some arguments. Rewrite the function using reference parameters. Again test this function with some arguments.

```

#include<iostream>
using namespace std;

void Swap1(string x,string y){
    string temp = x;
    x = y;
    y = temp;
}

void Swap2(string& x,string& y){
    string temp = x;
    x = y;
    y = temp;
}

```

```

int main(){
    string a="Ant",b="Cat";
    cout<<"Before a:"<<a<<" b:"<<b<<endl;
    Swap1(a,b);
    cout<<"After Swap1 a:"<<a<<" b:"<<b<<endl;
    Swap2(a,b);
    cout<<"After Swap2 a:"<<a<<" b:"<<b<<endl;
    return 0;
}

```

Constants and references

1. Write a function that takes an integer and returns the factorial of that number. Declare function parameter as read only reference. Call the function with some argument from main function, store the result and print it.

```

#include<iostream>
using namespace std;

int fact(const int& n){
    if(n==0) return 1;
    else return n*fact(n-1);
}

int main(){
    int n;
    cin>>n;
    int f = fact(n);
    cout<<f;
    return 0;
}

```

Constants and pointers

1. Write a function Strcpy to copy one string to another with suitable formal parameters declarations. Following points must be considered.
 - a) Source string must not get modified
 - b) Target string is allowed to get modified
 - c) The Pointers must be constant pointers. Use it to copy some strings.

```
#include<iostream>
using namespace std;
void strcpy(string &str, char* ch)
{
    int i;
    for(i = 0; i < str.length(); i++)
        *(ch+i) = str[i];
    *(ch+i) = '\0';
}

int main()
{
    string s = "InformationTechnology";
    char ch[s.length()+1];
    strcpy(s,ch);
    cout << ch;
    return 0;
}
```

Inline function

1. Write an inline function add() that takes three integer arguments and returns the sum of these arguments.

```
#include<iostream>
using namespace std;

inline int Add(int x,int y,int z){
    return (x+y+z);
}

int main()
{
    int p = Add(3,2,1);
    cout<<p;
    return 0;
}
```

Function Overloading

1. Consider the following two scenarios:
 - a) We want to find out the maximum between three integers.

b) We also want to find out the maximum element of an array of integers.
Write two overloaded functions for these two scenarios.

```
#include<iostream>
using namespace std;

int Max(int x,int y,int z){
    if(x>y){
        if(z>x) return z;
        else return x;
    }
    else{
        if(z>y) return z;
        else return y;
    }
}

int Max(int *arr,int n){
    int m = arr[0];
    for(int i=1;i<n;i++){
        if(arr[i]>m) m=arr[i];
    }
    return m;
}

int main()
{
    int p = Max(3,2,1);
    cout<<p<<endl;
    int a[] = {3,8,2,7};
    int q = Max(a,4);
    cout<<q<<endl;
    return 0;
}
```

2. Write two overloaded functions print() such that one prints the elements of a vector and the other prints elements of a matrix. Note that a vector and a matrix may be represented as a one-dimensional array and a two-dimensional array respectively.

```
#include<iostream>
using namespace std;

void Print(int *arr,int n){
    for(int i=0;i<n;i++){
        cout<<arr[i]<<" ";
    }
    cout<<endl;
}

void Print(int *arr,int m,int n){
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
```

```

        cout<<*(arr+i*n+j)<<" ";
    }
    cout<<endl;
}
}
int main()
{
    int a[4] = {3,7,8,6};
    int b[3][2] = {{1,3},{2,4},{5,7}};
    Print(a,4);
    Print(&b[0][0],3,2);
    return 0;
}

```

Default values for function parameters

1. Consider function add() in 13. Specify the default values for second and third parameters to 0 (zero). Now call this function with three, two and one arguments and see the result.

```

#include<iostream>
using namespace std;

int Add(int x,int y=0,int z=0){
    return (x+y+z);
}
int main()
{
    cout<<Add(3,2,1)<<endl;
    cout<<Add(3,2)<<endl;
    cout<<Add(3)<<endl;
    return 0;
}

```